

Métodos de Computação Experimental e Quantitativa

Aula 03: **Formulação de Hipóteses e Variáveis**

Prof. Dr. Leonardo N. Ferreira e Prof. Dr. Wemerson D. Parreira

Conteúdo

1. Perguntas de Pesquisa e a Abordagem GQM
2. Formulação de Hipóteses (H_0 e H_1)
3. Classificação e Taxonomia de Variáveis
4. Atividade Prática: Elaboração da Ficha de Hipótese

O que sabemos das aulas anteriores

Para planejar experimentos, precisamos:

1. Problema bem definido
2. Hipótese testável

Por que isso importa em Eng. de Software?

- Decisões em Eng. de Software costumam ser baseadas em opinião, moda ou autoridade.
- A **Evidence-based software engineering (EBSE)**[1] propõe:

EBSE

Tratar escolhas como afirmações testáveis com dados coletados sistematicamente.

Aula de Hoje

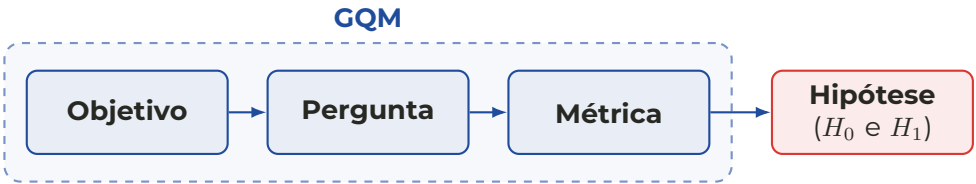
Transformar dúvidas práticas de engenharia em uma hipótese testável.

[1] Barbara Ann Kitchenham, Tore Dybå e Magne Jørgensen. "Evidence-based software engineering". Em: *Proceedings of the 26th International Conference on Software Engineering (ICSE '04)*. Washington, DC, USA: IEEE Computer Society, 2004, pp. 273–281.

Goal-Question-Metric Framework

Goal-Question-Metric[2]

Framework para definir objetivos de pesquisa e métricas associadas.



Importante

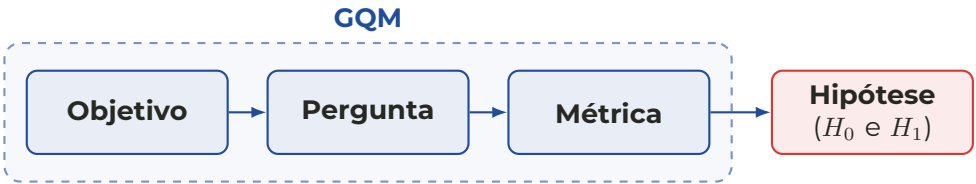
Sem uma hipótese formal, a métrica não sustenta decisões baseadas em evidência.

[2]Victor R. Basili e H. Dieter Rombach. “The TAME project: Towards improvement-oriented software environments”. Em: *IEEE Transactions on Software Engineering* 14.6 (1988), pp. 758–773.

Goal-Question-Metric Framework

Goal-Question-Metric[2]

Framework para definir objetivos de pesquisa e métricas associadas.



Importante

Sem uma hipótese formal, a métrica não sustenta decisões baseadas em evidência.

[2] Basili e Rombach, "The TAME project: Towards improvement-oriented software environments".

GQM aplicado — Exemplo

Objetivo

Analisar a adoção de **análise estática de código**, com o propósito de **melhorar a confiabilidade**, do ponto de vista da **equipe de qualidade**, no contexto de **projetos de médio porte**.

Pergunta

O uso de análise estática reduz a densidade de defeitos que chegam à fase de Garantia de Qualidade (QA – *Quality Assurance*)?

Métrica

Densidade de defeitos reportados em QA por KLOC em projetos com e sem a ferramenta.

Hipótese (o passo de hoje)

H_0: não há diferença na densidade de defeitos em QA entre projetos com e sem análise estática.
H_1: projetos com análise estática apresentam menor densidade de defeitos em QA.

Perguntas de Pesquisa Modelo

Modelo

Em ***[contexto]***, o uso de ***[intervenção]*** está associado a ***[efeito esperado]*** em ***[métrica]***, comparado a ***[baseline]***?

Exemplos:

- **Contextos:** projetos legados, projetos novos, equipes pequenas, equipes grandes, etc.
- **Intervenções:** adoção de uma ferramenta, adoção de uma técnica, adoção de uma metodologia, etc.
- **Efeitos esperados:** redução de custo, aumento de produtividade, melhoria da qualidade, etc.
- **Métricas:** custo, produtividade, qualidade, etc.
- **Baseline:** prática atual, prática anterior, prática de referência, etc.

Perguntas de Pesquisa Modelo

Modelo

Em ***[contexto]***, o uso de ***[intervenção]*** está associado a ***[efeito esperado]*** em ***[métrica]***, comparado a ***[baseline]***?

Exemplos:

- **Contextos:** projetos legados, projetos novos, equipes pequenas, equipes grandes, etc.
- **Intervenções:** adoção de uma ferramenta, adoção de uma técnica, adoção de uma metodologia, etc.
- **Efeitos esperados:** redução de custo, aumento de produtividade, melhoria da qualidade, etc.
- **Métricas:** custo, produtividade, qualidade, etc.
- **Baseline:** prática atual, prática anterior, prática de referência, etc.

Perguntas de Pesquisa

Do vago ao delimitado

Vago (não serve)

“Pair programming é bom?”

Delimitado (pergunta de pesquisa)

“Em equipes de desenvolvedores juniores, a adoção de ***pair programming*** reduz a **densidade de defeitos por KLOC** em comparação ao **desenvolvimento individual**?”

Vago (não serve)

“Testes automatizados ajudam?”

Delimitado (pergunta de pesquisa)

“Em projetos de médio porte, o uso de testes automatizados reduz o tempo de detecção de defeitos em relação a testes manuais?”

Perguntas de Pesquisa

Do vago ao delimitado

Vago (não serve)

“Pair programming é bom?”

Delimitado (pergunta de pesquisa)

“Em equipes de desenvolvedores juniores, a adoção de ***pair programming*** reduz a **densidade de defeitos por KLOC** em comparação ao **desenvolvimento individual**?”

Vago (não serve)

“Testes automatizados ajudam?”

Delimitado (pergunta de pesquisa)

“Em projetos de médio porte, o uso de testes automatizados reduz o tempo de detecção de defeitos em relação a testes manuais?”

Perguntas de Pesquisa Devem ser

- **Delimitada**: escopo claro (contexto, população e comparação).
- **Testável/Falseável**: passível de refutação por dados empíricos.
- **Relevante**: conecta-se a uma decisão prática de engenharia.
- **Sem viés**: não pressupõe o resultado esperado (não retórica).

Exercício 1

Enunciado

Transformem cada pergunta vaga abaixo em uma pergunta de pesquisa delimitada, seguindo a estrutura apresentada:

a) “Assistentes de IA generativa ajudam a programar melhor?”

b) “Testes de mutação valem o esforço de manutenção?”

c) “Revisão de código por pares (*code review*) melhora a qualidade do software?”

d) “A migração para microsserviços melhora a escalabilidade do sistema?”

HIPÓTESES

Hipótese

Definição

Resposta provisória e testável ao problema, formulada antes da coleta de dados.

Hipótese nula (H_0)

Assume que **não há efeito** ou diferença. É o estado padrão.

Hipótese alternativa (H_1)

Afirma que **há diferença** ou efeito. É a tese que buscamos evidenciar.

Notação estatística[3]

Considere μ a média populacional da variável de resposta.

Bilateral – sem direção definida

$H_0 : \mu_A = \mu_B \qquad H_1 : \mu_A \neq \mu_B$

Unilateral – direção hipotetizada

$H_0 : \mu_A = \mu_B \qquad H_1 : \mu_A < \mu_B \quad \text{ou} \quad \mu_A > \mu_B$

Importante

A escolha da hipótese depende do conhecimento prévio sobre a direção do efeito.

[3] Claes Wohlin et al. *Experimentation in Software Engineering*. Springer, 2012.

Exemplo

H_0 (em palavras)

Não há diferença na densidade de defeitos entre pair programming e individual.

H_1 (em palavras)

Pair programming **reduz** a densidade de defeitos (unilateral).

Notação formal

Com μ_{pares} e $\mu_{individual}$ representando as médias sob cada tratamento:

$$H_0 : \mu_{pares} = \mu_{individual} \qquad H_1 : \mu_{pares} < \mu_{individual}$$

Por que Média Populacional (μ) e não Amostral ($\bar{x}$)?

Hipóteses são afirmações sobre a população (comportamento geral), não sobre a amostra.

- **Média Populacional (μ)**: parâmetro geral desconhecido.
  $\rightarrow$ Formulamos as hipóteses (H_0 e H_1) sobre ela.
- **Média Amostral ($\bar{x}$)**: valor exato calculado dos dados coletados.
  $\rightarrow$ Não há dúvida sobre ela, por isso não criamos hipóteses.

Raciocínio

Usamos a amostra ($\bar{x}$) como evidência para tirar conclusões sobre a população (μ).

Por que tentamos refutar H_0 e não provar H_1?

- **Assimetria lógica (Falseabilidade)**:
  Dados amostrais **nunca provam** H_1 de forma definitiva, mas podem fornecer evidências suficientes para **rejeitar** H_0.

- **Ausência de evidência $\neq$ Evidência de ausência**:
  Não rejeitar H_0 **não prova** que H_0 é verdadeira; apenas indica que não há dados suficientes para descartá-la.

Em resumo

Não "provamos" H_1; rejeitamos o acaso (H_0) como explicação plausível para a diferença observada.

Exercício 2 formule H_0 e H_1

Enunciado

Para cada pergunta de pesquisa abaixo, escreva H_0 e H_1:

1. “O uso de tipagem estática reduz o número de bugs em relação à tipagem dinâmica?”

2. “Equipes que fazem *code review* entregam código com menos vulnerabilidades de segurança?”

3. “A migração para arquitetura de microsserviços aumenta o tempo médio de resposta do sistema?”

Exercício 2 Respostas

1. H_0: não há diferença no num. de bugs entre tipagem estática e dinâmica.
   H_1: tipagem estática está associada a menor número de bugs.

2. H_0: não há diferença na quantidade de vulnerabilidades entre equipes com e sem *code review*.
   H_1: equipes com *code review* apresentam menos vulnerabilidades.

3. H_0: não há diferença no tempo médio de resposta entre as arquiteturas.
   H_1: a arquitetura de microsserviços está associada a maior tempo médio de resposta.

Atenção

H_0 e H_1 precisam ser **mutuamente exclusivas e exaustivas**

VARIÁVEIS: REVISÃO E FORMALIZAÇÃO

Tipos de variáveis Lembrando

Variável Independente (VI) ou Tratamento

O que é manipulado ou comparado.
Exemplo: uso ou não de pair programming

Variável Dependente (VD) ou Resposta

O que é medido como resultado.
Exemplo: número de defeitos por KLOC

Variável de controle

Mantida constante para não confundir o efeito da variável independente.
Exemplo: mesma linguagem, mesma complexidade de tarefa

Variável moderadora

Altera a força ou a direção do efeito da VI sobre a VD.
Exemplo: nível de experiência: júnior vs. sênior

Taxonomia de variáveis de resposta

Categorias de variáveis dependentes (VD) em Engenharia de Software:

Categoria	Exemplos	Pergunta típica
Produto	Defeitos, cobertura, desempenho	“É mais correto/rápido?”
Processo	Esforço, tempo, custo	“É mais barato/rápido de fazer?”
Pessoas	Satisfação, carga cognitiva	“É mais fácil de usar/entender?”

Dica: É comum combinar VDs de diferentes categorias em um único experimento.

Exemplo programação em pares vs. individual

Papel	Exemplo no caso de Pares
VI	Estratégia de desenvolvimento (pares vs. individual)
VD	Densidade de defeitos / tempo de entrega
Controle	Linguagem, complexidade da tarefa
Moderadora	Nível de experiência (júnior/sênior)

Exercício 3

Enunciado

Uma equipe testa se o uso de IA generativa para gerar testes unitários reduz o tempo de desenvolvimento, e desconfia que esse efeito é diferente para desenvolvedores júniores e sêniores. Identifique:

- Variável Independente (VI):
- Variável Dependente (VD):
- Variável de Controle (VC):
- Variável Moderadora (VM):

TRABALHO EM GRUPO

Trabalho em grupo Enunciado

Passo a passo (em equipe):

1. Escolham um tema de interesse.
2. Redijam a pergunta de pesquisa (abordagem GQM).
3. Formulem H_0 e H_1 usando notação μ_A / μ_B.
4. Classifiquem as variáveis (VI - fator, VD - resposta, controle, moderadora).
5. Preencham a Ficha de Hipótese.

Nota

A pergunta é um rascunho de trabalho para refinar após a revisão de literatura.

Trabalho em grupo

Preencher ficha de hipótese

Modelo

```
PROJETO: _____________________________________________

PROBLEMA (lacuna de conhecimento):
_____________________________________________________

PERGUNTA DE PESQUISA:
_____________________________________________________

H0 (hipótese nula):
_____________________________________________________

H1 (hipótese alternativa):   [ ] unilateral   [ ] bilateral
_____________________________________________________

VI:                     VD:
CONTROLE:               MODERADORA (se houver):
```

Trabalho em grupo

Critérios de qualidade

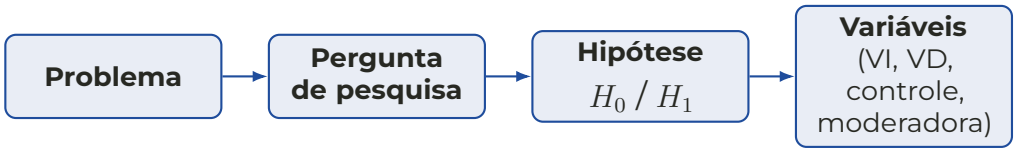
Checklist de avaliação do trabalho:

- ☐ Pergunta delimitada (contexto e comparação explícitos).
- ☐ Hipóteses H_0 e H_1 mutuamente exclusivas.
- ☐ Variável Independente passível de manipulação/comparação.
- ☐ Variável Dependente mensurável (com unidade quantitativa).
- ☐ Pelo menos uma variável de controle.

Entregar no Canvas!

Resumo da Aula

Fluxo de Formulação Experimental:



Mensagem central

Uma hipótese científica não é opinião. É uma afirmação testável e falseável, vinculada a variáveis quantificáveis.

Referências Bibliográficas I

- Basili, Victor R. e H. Dieter Rombach. “The TAME project: Towards improvement-oriented software environments”. Em: *IEEE Transactions on Software Engineering* 14.6 (1988), pp. 758–773.
- Juristo, Natalia e Ana M. Moreno. *Basics of Software Engineering Experimentation*. Boston, MA: Kluwer Academic Publishers, 2001.
- Kitchenham, Barbara Ann, Tore Dybå e Magne Jørgensen. “Evidence-based software engineering”. Em: *Proceedings of the 26th International Conference on Software Engineering (ICSE ’04)*. Washington, DC, USA: IEEE Computer Society, 2004, pp. 273–281.
- Wohlin, Claes et al. *Experimentation in Software Engineering*. Springer, 2012.